

POCKETSMART AI

8 Project Development Phases – Explanation & Separate Code

Project: PocketSmart AI – Your Smart Budget & Recommendation Assistant

Technology: Python, FastAPI, HTML/CSS/JavaScript, Gemini AI, Jinja2

Purpose: This document organizes the project into eight development phases. Each phase contains its objective, activities, expected output, and a separate sample code section.

1. Brainstorming & Ideation Phase

Explanation: The brainstorming phase identifies the problem PocketSmart AI will solve and converts the idea into clear modules. The project focuses on budget-aware recommendations for Home Interior, Party Planning, and Jewelry selection. Users provide budget, preferences, event/occasion details, and optionally an outfit image. The AI then produces personalized suggestions.

Separate Phase Code:

```
# brainstorming.py
project_name = "PocketSmart AI"

ideas = [
    "Home Interior Budget Planner",
    "Party Budget Planner",
    "Jewelry Budget Planner",
    "AI-based recommendations",
    "Budget tracking and history"
]

for idea in ideas:
    print("Idea:", idea)

print("\nSelected concept:", project_name)
print("Goal: Give personalized recommendations within a user's budget.")
```

Expected Output: Output: A clearly defined project concept, three main planner modules, and a list of core features.

2. Requirement Analysis Phase

Explanation: Requirement analysis converts the idea into functional and non-functional requirements. Functional requirements include user registration/login, budget input, Home/Party/Jewelry planners, optional image upload for jewelry, AI recommendations, recommendation history, and session handling. Non-functional requirements include usability, responsiveness, modularity, security, and reliable API communication.

Separate Phase Code:

```
# requirements.py
requirements = {
    "functional": [
        "User registration and login",
        "Home budget planner",
        "Party budget planner",
        "Jewelry budget planner",
        "AI recommendations",
        "Recommendation history"
    ],
    "non_functional": [
        "Responsive UI",
        "Secure authentication",
        "Modular backend",
        "Fast API communication"
    ]
}

for category, items in requirements.items():
    print("\n" + category.upper())
    for item in items:
        print("-", item)
```

Expected Output: Output: A structured requirement list that can be used as the development checklist.

3. Project Design Phase

Explanation: The design phase defines the architecture and data flow. The source document describes a frontend connected to a FastAPI backend, with a Gemini AI layer for recommendation generation. The backend contains routes for Home, Party, Jewelry, authentication, sessions, history, and recommendation details. Jinja2 templates are used for the frontend pages.

Separate Phase Code:

```
# project_design.py
architecture = {
    "Frontend": ["HTML", "CSS", "JavaScript", "Jinja2"],
    "Backend": ["FastAPI", "API Routes", "Session/Auth"],
    "AI Layer": ["Gemini AI", "Prompt Processing"],
    "Modules": ["Home", "Party", "Jewelry"],
    "Data": ["User Session", "Recommendation History"]
}

for layer, components in architecture.items():
    print(f"\n{layer}:")
    for component in components:
        print("  ->", component)
```

Expected Output: Output: A logical architecture showing how the frontend, backend, AI layer, and planner modules connect.

4. Project Planning Phase

Explanation: Project planning divides development into manageable tasks and milestones. The PocketSmart AI source organizes work around AI initialization, core functionality, backend implementation, UI development, and testing/optimization. Planning also includes deciding file structure, API routes, dependencies, test cases, and development order.

Separate Phase Code:

```
# project_plan.py
tasks = [
    ("Phase 1", "Brainstorming & Ideation"),
    ("Phase 2", "Requirement Analysis"),
    ("Phase 3", "Project Design"),
    ("Phase 4", "Project Planning"),
    ("Phase 5", "Project Development"),
    ("Phase 6", "Project Testing"),
    ("Phase 7", "Project Documentation"),
    ("Phase 8", "Project Demonstration")
]

for phase, task in tasks:
    print(f"{phase}: {task}")

print("\nDevelopment order planned successfully.")
```

Expected Output: Output: A phase-wise development schedule and task checklist.

5. Project Development Phase

Explanation: Development implements the planned system. The project source uses FastAPI routes such as /generate-home, /generate-party, /generate-jewelry, /register, /login, /logout, /token, /session-info, /session-data, /recommendations-details, and /history. The backend communicates with the Gemini recommendation layer and the frontend renders results.

Separate Phase Code:

```
# main.py
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="PocketSmart AI")

class PlannerRequest(BaseModel):
    budget: float
    preference: str = ""

@app.get("/")
def home():
    return {"message": "PocketSmart AI is running"}

@app.post("/generate-home")
def generate_home(data: PlannerRequest):
    return {
        "planner": "Home",
        "budget": data.budget,
        "preference": data.preference,
        "recommendation": "Generate budget-aware home recommendations"
    }

@app.post("/generate-party")
def generate_party(data: PlannerRequest):
    return {"planner": "Party", "budget": data.budget}

@app.post("/generate-jewelry")
def generate_jewelry(data: PlannerRequest):
    return {"planner": "Jewelry", "budget": data.budget}
```

Ex

CO

```
from fastapi import APIRouter
from app.services.recommendation_service
import get_recommendation

router = APIRouter()

@router.get("/recommendations-details")
def recommendations_details(category: str,
    details: str):
    result = get_recommendation(category,
    details)

    return {
        "category": category,
        "recommendation": result
    }
```

6. Project Testing Phase

Explanation: Testing checks whether the system works correctly for different budgets, preferences, planner types, authentication states, and optional image inputs. The source specifically recommends end-to-end testing for Home decor, Party packages, and Jewelry suggestions, along with checking AI response quality, budget adherence, platform accuracy, validations, and fallback recommendations.

Separate Phase Code:

```
# test_api.py
def validate_budget(budget):
    return isinstance(budget, (int, float)) and budget > 0

def test_valid_budget():
    assert validate_budget(10000) is True

def test_invalid_budget():
    assert validate_budget(0) is False
    assert validate_budget(-500) is False

def test_planner_names():
    planners = ["Home", "Party", "Jewelry"]
    assert len(planners) == 3

if __name__ == "__main__":
    test_valid_budget()
    test_invalid_budget()
    test_planner_names()
    print("All basic tests passed.")
```

Expected Output: Output: Test results showing whether basic validation and planner logic behave as expected.

```
from pydantic import BaseModel
```

```
class RecommendationRequest(BaseModel):
    category: str
    details: str
```

```
class RecommendationResponse(BaseModel):
    category: str
    recommendation: str
```

7. Project Documentation Phase

Explanation: Documentation records how the system works so that developers, evaluators, and users can understand and maintain it. It should include project overview, objectives, requirements, architecture, modules, API routes, setup steps, testing results, screenshots, limitations, and conclusion. The supplied project document already contains architecture, workflow, milestones, UI pages, testing, and conclusion sections.

Separate Phase Code:

```
# generate_documentation.py
sections = [
    "Project Overview",
    "Objectives",
    "Requirement Analysis",
    "System Architecture",
    "Module Description",
    "API Endpoints",
    "Installation and Setup",
    "Testing Results",
    "Screenshots",
    "Conclusion"
]

with open("PROJECT_DOCUMENTATION.txt", "w", encoding="utf-8") as file:
    for number, section in enumerate(sections, 1):
        file.write(f"{number}. {section}\n")

print("PROJECT_DOCUMENTATION.txt created successfully.")
```

Expected Output: Output: A documentation file containing the main headings required for the project report.

```
from fastapi import APIRouter
from app.services.auth_service import
hash_password, verify_password

router = APIRouter()

users = {}

@router.post("/register")
def register(username: str, password: str):
    if username in users:
        return {"message": "User already
exists"}

    users[username] =
hash_password(password)

    return {"message": "Registration
successful"}

@router.post("/login")
def login(username: str, password: str):
    if username not in users:
        return {"message": "User not found"}

    if not verify_password(password,
users[username]):
        return {"message": "Invalid
password"}

    return {"message": "Login successful"}
```

8. Project Demonstration Phase

Explanation: The demonstration phase presents the completed PocketSmart AI to the evaluator. A typical demonstration flow is: start the FastAPI server, open the home page, register/login, select a planner, enter budget and preferences, submit the request, display AI recommendations, and show recommendation history. For the Jewelry planner, demonstrate the optional outfit image input if that feature is configured.

Separate Phase Code:

```
# demo_checklist.py
demo_steps = [
    "Start FastAPI server",
    "Open PocketSmart AI home page",
    "Register or login",
    "Open Home/Party/Jewelry planner",
    "Enter budget and preferences",
    "Submit planner request",
    "Display recommendations",
    "Show recommendation history"
]

print("POCKETSMART AI - DEMO CHECKLIST")
for i, step in enumerate(demo_steps, 1):
    print(f"{i}. {step}")

print("\nDemo flow completed.")
```

Expected Output: Output: A simple demonstration checklist that can be followed during the project viva/demo.

```
from fastapi import FastAPI
from fastapi.templating import Jinja2Templates
templates = Jinja2Templates(directory="app/templates")
from fastapi import Request

import os

from app.routes import (
    home,
    party,
    jewelry,
    auth,
    session,
    recommendations,
    history,
    startup
)

app = FastAPI(title="PocketSmart: AI Budget Planner")
templates = Jinja2Templates(directory="app/templates")

SECRET_KEY = os.getenv("SECRET_KEY", "your_secret_key")

app.include_router(home.router)
app.include_router(party.router)
app.include_router(jewelry.router)
app.include_router(auth.router)
app.include_router(session.router)
app.include_router(recommendations.router)
app.include_router(history.router)
app.include_router(startup.router)

@app.get("/")
def home_page(request: Request):
    return templates.TemplateResponse(
        request=request,
        name="index.html",
        context={}
    )
```

Final Project Flow

Phase	Main Result
1. Ideation	Project concept and features
2. Requirements	Functional and non-functional requirements
3. Design	Architecture and module/data flow
4. Planning	Tasks, milestones and development order
5. Development	Working frontend/backend/AI modules
6. Testing	Validated functions and user scenarios
7. Documentation	Complete project report and technical notes
8. Demonstration	Live project presentation and walkthrough

Source alignment: The phase descriptions and technical details are based primarily on the uploaded PocketSmart AI project document, including its FastAPI backend, Gemini AI integration, three planner modules, UI development, testing/optimization, and project conclusion.
filecite turn0 L427-L460